

TP5 – Communications Réseaux et Services Web

Objectif du TP

L'objectif de ce TP est de transformer l'application **ToDoList** réalisée lors des TP3 et TP4 en une application connectée à un **service Web distant**.

Vous allez utiliser **Retrofit**, une bibliothèque Android moderne, pour effectuer des requêtes HTTP et consommer des **APIs REST**.

Pré-requis

- Android Studio (version récente)
 - Le projet des TP3 et TP4 avec deux activités :
 - **MainActivity** (authentification)
 - **ToDoActivity** (affichage de la liste)
 - Connexion Internet
 - APIs distantes :
 - <https://belatar.info/rest/login.php> (*login/passwd = test/test ou test1/test1*)
 - <https://belatar.info/rest/getTasks.php>
 - <https://belatar.info/rest/editTask.php> (*pour la partie bonus*)
-

1. Étapes de réalisation

1.1. Importation des dépendances Retrofit

Dans le fichier **build.gradle (Module)** :

```
implementation 'com.squareup.retrofit2:retrofit:3.0.0'  
implementation 'com.squareup.retrofit2:converter-gson:3.0.0'
```

Synchroniser le projet après ajout.

1.2. Déclaration des permissions réseau

Dans le fichier `AndroidManifest.xml` :

```
<uses-permission android:name="android.permission.INTERNET" />
```

Cette permission est nécessaire pour permettre à l'application d'accéder à internet afin de consommer les APIs REST.

1.3. Modèle de données

Créer une classe `TodoTask.java` représentant une tâche :

```
public class TodoTask {  
    private int id;  
    private String desc;  
    private int done;  
    ///TODO: constructeur, getters, setters  
}
```

1.4. Modèle de réponse JSON

L'API `getTasks.php` renvoie un objet contenant un tableau `tasks` et éventuellement un message d'erreur.
On crée donc une classe `TasksResponse.java` :

```
public class TasksResponse {  
    private List<TodoTask> tasks;  
    ///TODO: getters, setters  
}
```

De même, la réponse à l'authentification sera représentée par `LoginResponse.java` :

```
public class LoginResponse {  
    private int id;  
    private String nom;  
    private String prenom;  
    private String sessionToken;  
    ///TODO: getters, setters  
}
```

1.5. Interface de service Retrofit

Créer une interface `ApiService.java` pour définir les requêtes réseau :

```
public interface ApiService {
    @FormUrlEncoded
    @POST("login.php")
    Call<LoginResponse> login(
        @Field("login") String login,
        @Field("passwd") String passwd
    );

    @FormUrlEncoded
    @POST("getTasks.php")
    Call<TasksResponse> getTasks(
        @Field("sessionToken") String token
    );
}
```

1.6. Création de l'instance Retrofit

Créer une classe `ApiClient.java` :

```
public class ApiClient {
    private static final String BASE_URL = "https://belatar.info/rest/";
    private static Retrofit retrofit = null;

    public static ApiService getService() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
        return retrofit.create(ApiService.class);
    }
}
```

1.7. Authentification (MainActivity)

Lors du clic sur le bouton "Se connecter" :

1. Récupérer le login et le mot de passe.
2. Appeler l'API `login.php`.
3. Enregistrer les informations de session dans les `SharedPreferences`.

```
ApiService api = ApiClient.getService();
Call<LoginResponse> call = api.login(login, passwd);
call.enqueue(new Callback<LoginResponse>() {
    @Override
    public void onResponse(Call<LoginResponse> call, Response<LoginResponse> response) {
        if (response.isSuccessful()) {
            LoginResponse data = response.body();
            ///TODO: sauvegarder dans SharedPreferences et passer à l'autre activity
        } else {
            ///TODO: afficher Toast ou Snackbar
        }
    }

    @Override
    public void onFailure(Call<LoginResponse> call, Throwable t) {
        ///TODO: afficher Toast ou Snackbar
    }
});
```

1.8. Récupération et affichage des tâches (ToDoActivity)

1. Lire les informations `nom`, `prenom` et `token` depuis les `SharedPreferences`.
2. Afficher le nom complet dans la barre de titre.
3. Appeler `getTasks()` pour récupérer les tâches.
4. Afficher les tâches dans un `RecyclerView`.

```
ApiService api = ApiClient.getService();
Call<TasksResponse> call = api.getTasks(token);

call.enqueue(new Callback<TasksResponse>() {
    @Override
    public void onResponse(Call<TasksResponse> call, Response<TasksResponse> response) {
        if (response.isSuccessful()) {
            ///TODO: afficher les TodoTasks sur RecyclerView
        } else {
            ///TODO: afficher l'erreur retournée par l'API
        }
    }

    @Override
    public void onFailure(Call<TasksResponse> call, Throwable t) {
        Toast.makeText(ToDoActivity.this, "Erreur réseau", Toast.LENGTH_SHORT).show();
    }
});
```

1.9. Bonus (facultatif) : ajout, modification et suppression de tâches

Les étudiants qui souhaitent aller plus loin peuvent utiliser l'API suivante :

<https://belatar.info/rest/editTask.php>

Paramètres POST :

```
sessionToken=<token>

cmd=<add | del | edit>

description=<text> (si add ou edit)

done=<0 | 1> (si add ou edit)

id=<id_tache> (si edit ou del)
```

Réponses possibles :

```
{ "success": "Opération réussie" } ou { "error": "Message d'erreur" }
```

2. Résultats attendus

- Authentification distante fonctionnelle.
 - Récupération et affichage de la liste des tâches.
 - Gestion de la session via `SharedPreferences`.
 - Interface fluide et réactive (grâce à Retrofit).
 - (Bonus) Ajout, modification ou suppression de tâches via l'API.
-

3. Livrables attendus

- Code complet : `ApiClient`, `ApiService`, `LoginResponse`, `TasksResponse`, `MainActivity`, `ToDoActivity`, `ToDoAdapter`.
 - Captures d'écran :
 - Connexion réussie.
 - Liste des tâches affichées.
 - (Bonus) Ajout, suppression ou modification de tâches.
-